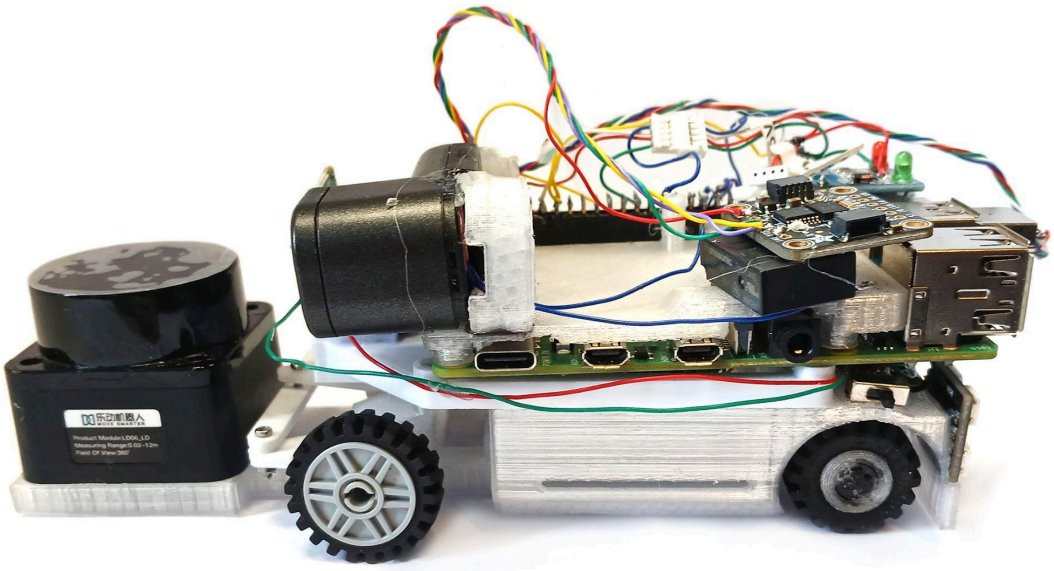
WRO 2025

ENGINEERING JOURNAL

Future Engineers



Team: NullPointerException

Contents

Introduction

Welcome to team NullPointerException's Engineering Journal. This document will guide you through our process of designing, building and programming our robot, along with demonstration videos of the final version, pictures, and step by step instructions for assembly.

This year's Self-Driving Car challenge features a reintroduction of the randomisation process, a removal of the need to turn around in the last round, and adjusted rules concerning parallel parking. Basing our current robot off its counterpart used in last year's challenge, this document will describe the revisions and reworks we have applied to comply with the added features of the challenge, as well as improve its overall functionality.

The Team

We are a South African team, made up of:

- Michael Shepstone (20) - studying Mechatronic Engineering
- Siobhan Kennedy (20) - studying Computational Linguistics
- Cody Williams - our coach

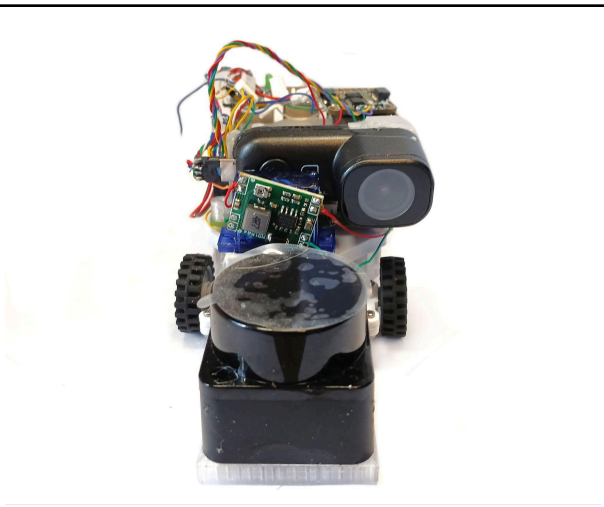
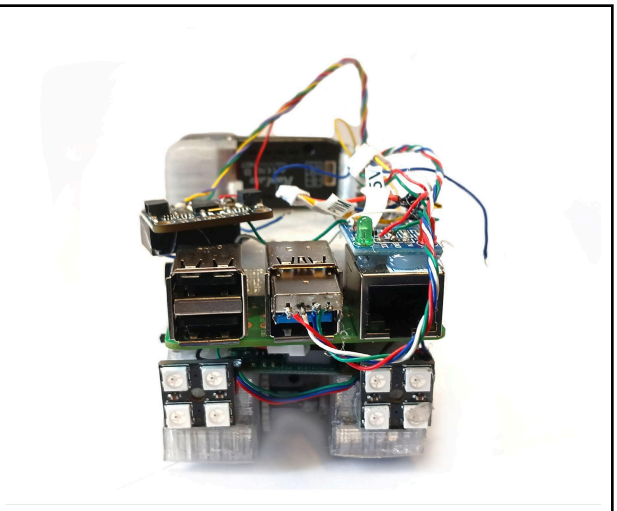
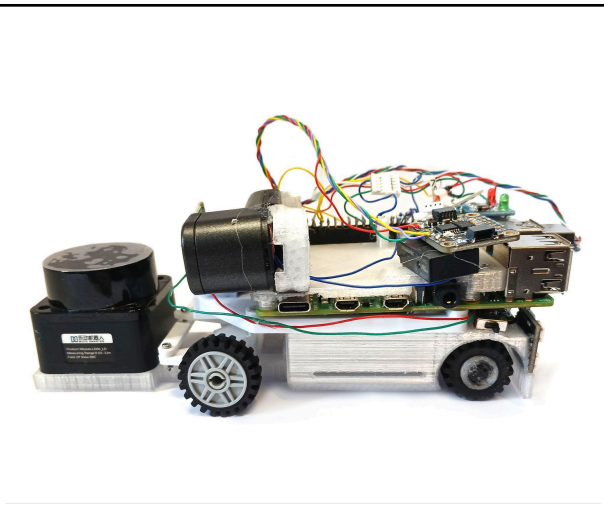
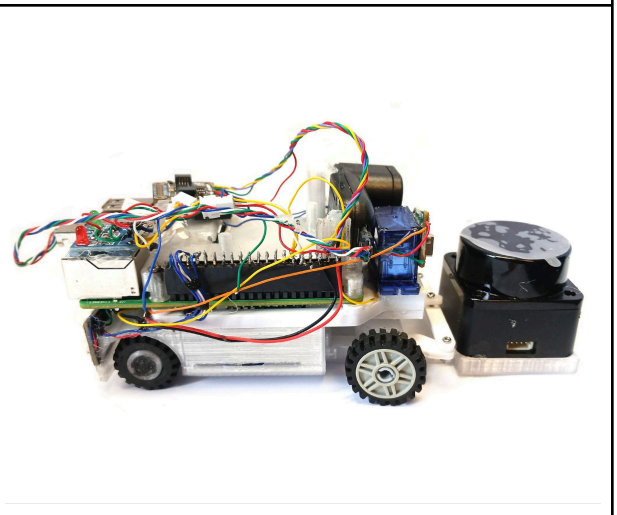
And of course, Chihuahua (the robot).

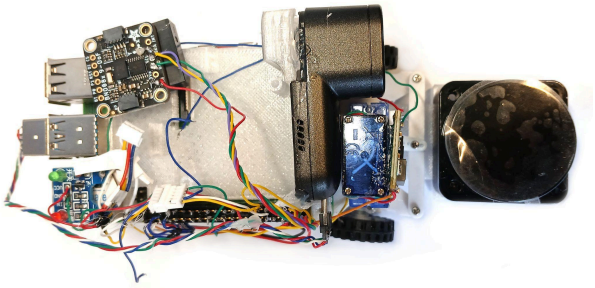
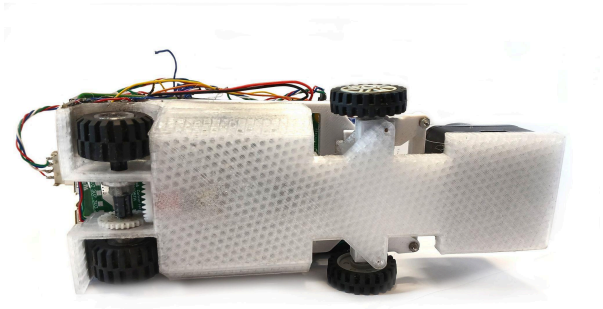
Chihuahua has seen many improvements and evolutions over the last three years of partaking in the World Robotics Olympiad's Future Engineers category, with this document describing its latest version, version 8.5.

Coming out of last year's competition, we brought numerous lessons and ideas on how we could improve our robot this time around, and ultimately decided to go back to the drawing board for the most part. In our mission to rehaul the robot, we discovered better and more efficient ways to make the robot function, while also learning several important lessons, which will be elaborated on throughout the document.

Siobhan Kennedy and Michael Shepstone

“Chihuahua” Mk8

	
Front view	*Back view*
	
Left side view	*Right side view*

	
Top view	*Bottom view*

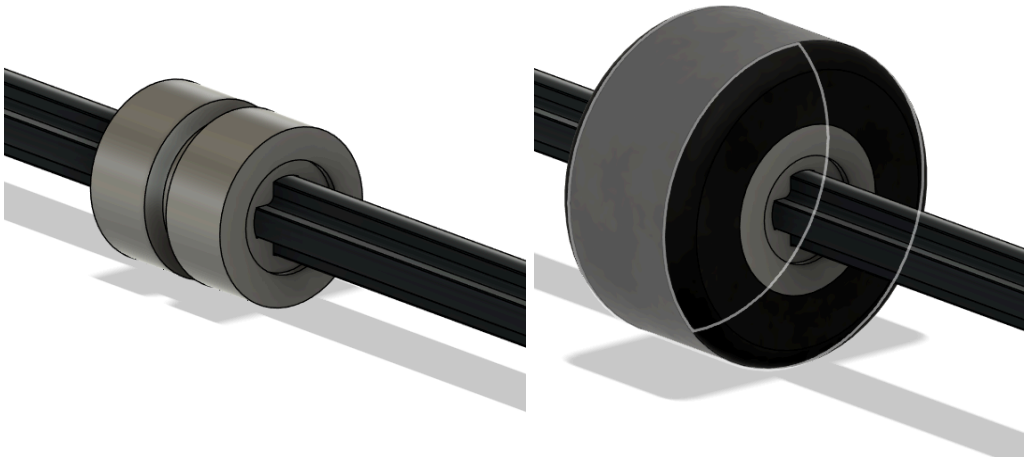
Mobility Management

POWERTRAIN

Wheels and Tyres

We used Lego rubber tyres (part number 60700) for the back wheels, and Lego rubber wheels and hubs (part number 61254 and 56902) for the front wheels. We learned that the wheels needed to be rubber for better grip and to avoid slipping. Lego makes rubber wheels that were perfectly suited to the adjustments we wanted to make.

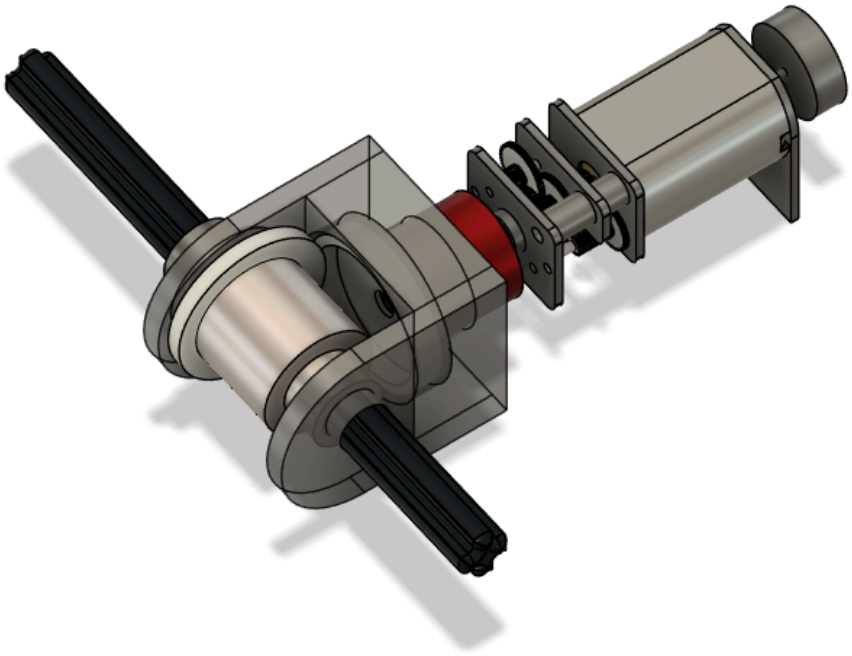
The tyres used for the back wheels were chosen due to their large width-to-diameter ratio, as well as their good grip. This ensures that the robot can remain sturdy while travelling across the surface and prevent any slipping. For whatever reason, lego makes no wheel hub for this particular tyre that can fit an L-series axle, thus we had to design and print our own.



Wheel adaptor on shaft, without and with tyre

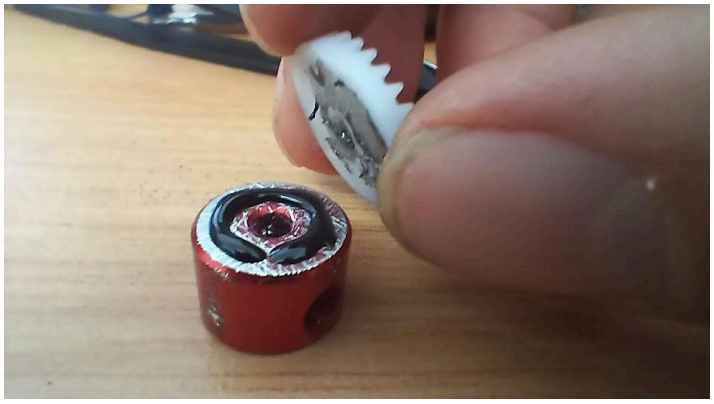
The tyres we used for the front wheels are relatively narrow, which causes high loading, which in turn reduces the clearance required for the robot to be able to steer smoothly. These wheels rotate freely.

Gearbox and Drivetrain



CAD Model of the complete motor-gearbox-axle assembly

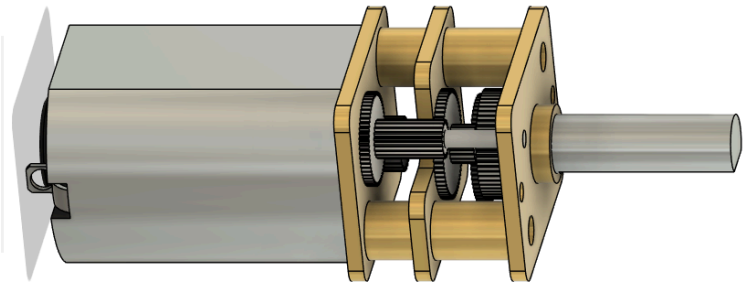
The robot's drivetrain currently consists of a lego L-series axle, and two gears that transfer the power by 90 degrees. To achieve this, the motor coupler has been cut in half, with the gear glued to it. This gear then meshes with a separate gear at a 90 degree angle to it. This second gear transfers the motion to the axle.



Attaching the gear to the halved motor coupler

Based on previous versions of the robot, we learned that ultimately, it was best to keep the wheel and axle system simple. We also previously attempted using a differential gearbox, but found that it was too large, and could easily cause problems should a wheel break ground contact.

Motor



After considering various different motors, we settled on an [N20 motor with an 800 rpm gearbox], with the following specifications:

• Voltage: 12V • Weight: 9.5g • Gear Ratio: 1:50 • Speed: 650 ± 31% rpm • Torque: 0.67 ± kg·cm	

Other motors we considered include: Micro Stepper Motors (precise and easy to drive, but too large and outside RPM range); 130 Size DC Motors (easy to obtain with pre-assembled attachments, but underpowered; and 25mm DC Motors (the motor we used in previous robots, but too large in this case). Ultimately, we decided on the N20 800rpm motor for its lightweight and compact design.

Other motors we considered include:

- Micro Stepper motors – Very precise, easy to drive with good power output, but was too large and outside the required RPM range.
    - Where to buy the motor:

      https://www.robotics.org.za/index.php?route=product/product&product_id=1438&search=stepper

- 130 Size DC Motors – Cheap and easy to obtain, with many pre-assembled attachments, but ended up being too underpowered and electronically noisy. Plus, it would have required an additional gearbox in any case.

    - Where to buy the motor:
      https://www.robotics.org.za/DC-MOTOR-6V?search=6v

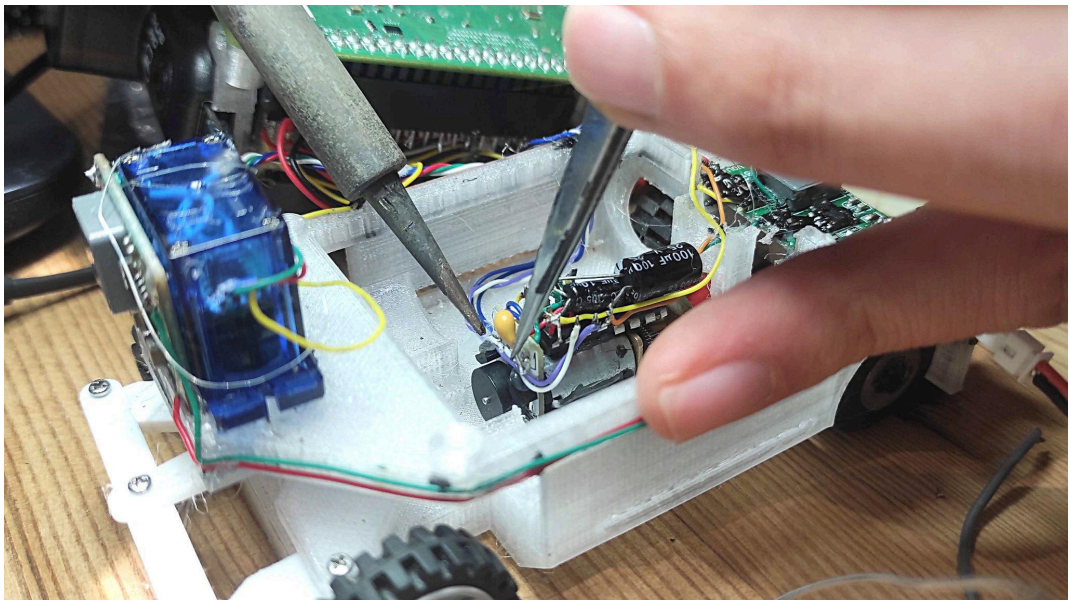
- 25mm DC Motors – Good power output with an included gearbox, the motor of choice for previous robots, but ended up being too large in this case.

    - Where to buy the motor:
      https://www.robotics.org.za/25GM370-060-12V?search=DC%20motor

Ultimately, we decided on the N20 800rpm motor for its lightweight and compact design. As this motor did not come with an encoder attached, we ended up using a 15rpm motor with the 800rpm motor transferred onto it.

Our encoder of choice is a Hall-effect magnetic encoder, which offers high precision and encodes direct motor output. In order to fit it into our robot, it had to be filed down, and the connector had to be removed with wires soldered directly to the PCB.

Filing down the encoder PCB



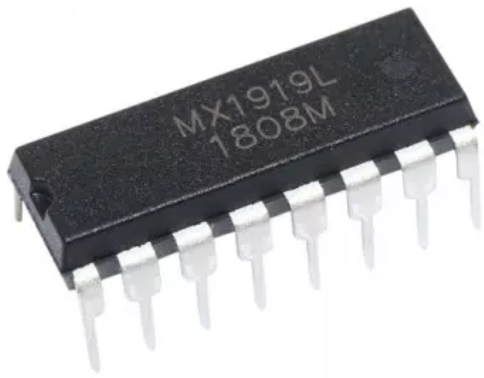
Soldering wires to the motor and motor driver

This motor connects directly to the aluminium motor coupler that is glued to the gear. The motor shaft is secured in place with a grub screw, instead of glue. Not only does this allow for a slight tolerance in the placement of the motor, but it also allows for parts to be swapped out, should it be necessary.

The previous iteration of this robot had a 3D printed driveshaft, which ended up proving to be problematic as it was prone to stripping or shearing. Therefore, based on the lessons learnt, all of our driveshaft are made of injection molded, or metallic components

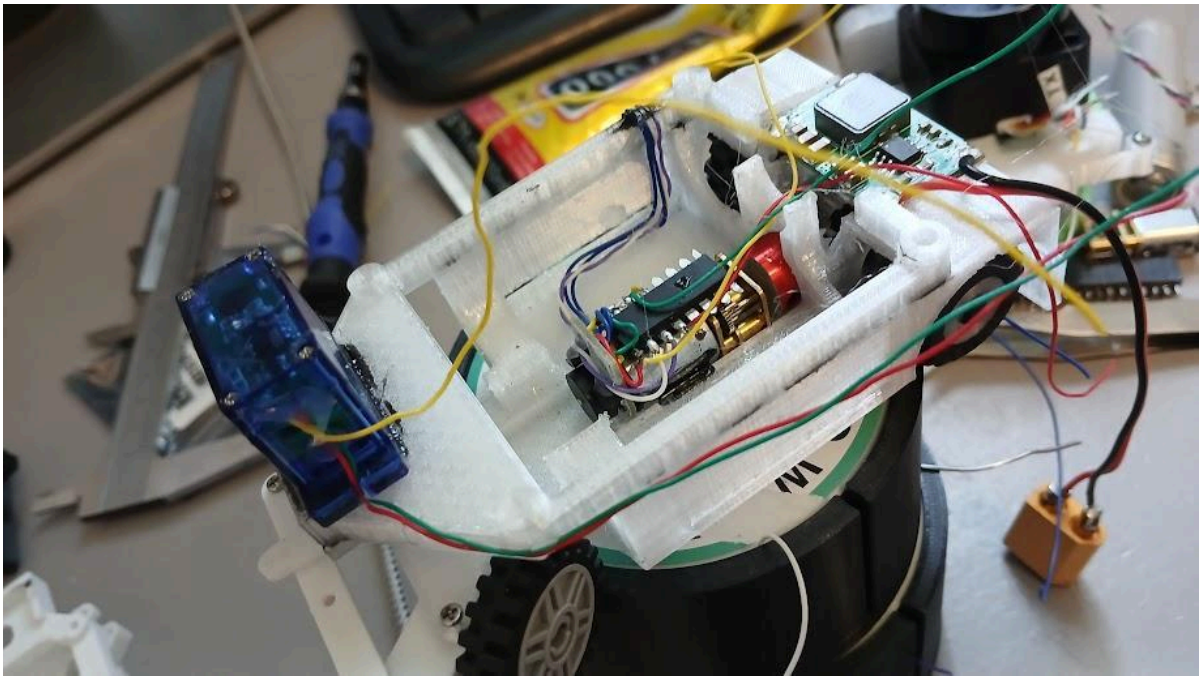
Motor Driver

For this iteration of the robot, we elected to use the MX1919 motor driver chip, with the following specifications:

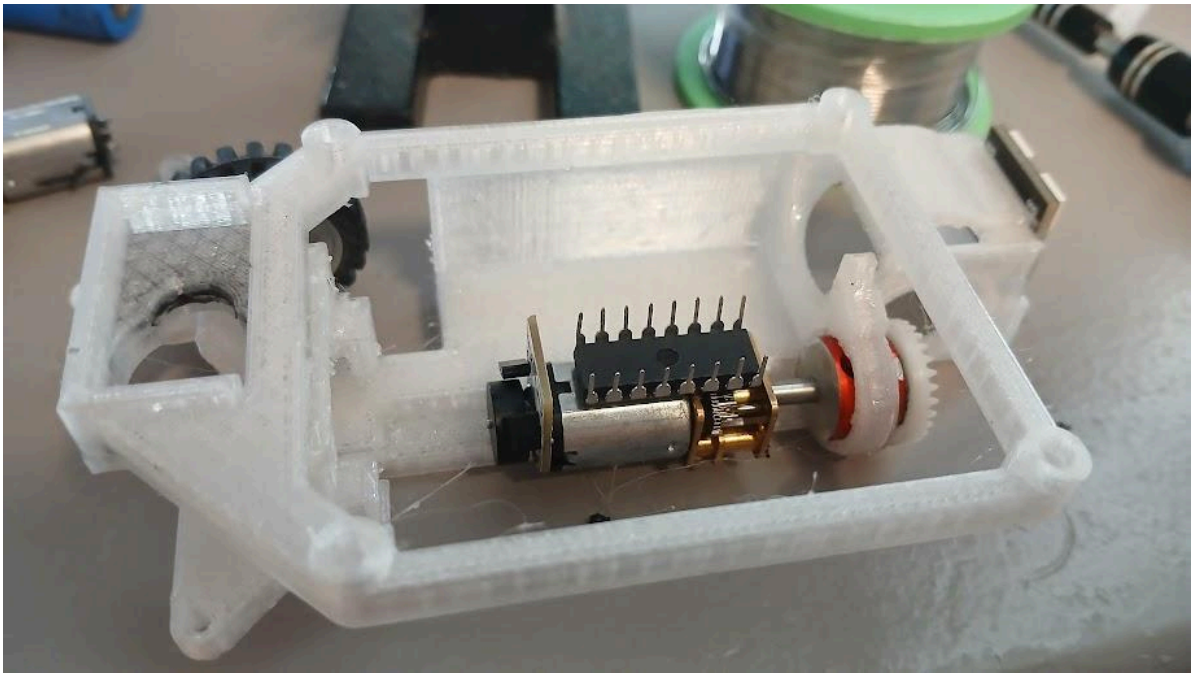
• Power Supply Voltage:
  ○ Min: 2V
  ○ Max: 9.6V
• Logic and Control Supply Voltage:
  ○ Min: 1.8V
  ○ Max: 5V
• Integrated Bridge Drive Circuit
	

The chip was attached to the motor backwards, and wires were soldered directly to the driver. Its feet were also clipped short for spatial efficiency

A very important lesson we learned from this part of the assembly process was that the motor driver must be wired with capacitors in order to prevent accidentally blowing the chip. We will not be disclosing how we discovered this, but you can probably guess.



Wires soldered to the motor driver



Motor driver mounted upside-down in demonstration frame

STEERING

Motor

For our steering motor, we opted to use the FS90 9g Analog Servo, with the following specifications:

• Closed loop (180 degrees) • Torque: 6V:1.5 kg·cm • Operating Voltage: 4.8V - 6V • Speed: 6V:0.10 sec/60° • Size: 23.2 * 12.5 * 22.0mm	

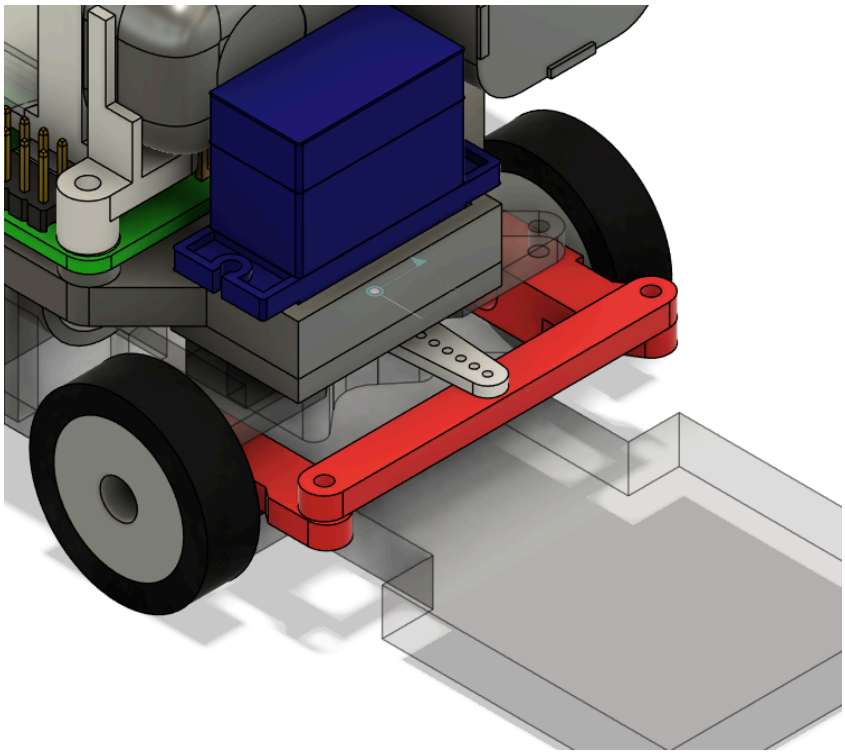
Ultimately, we chose this motor for its positional encoding. Additional factors include its cost efficiency, ample library support, and more-than-sufficient power output required to drive the steering system. At one point we considered using a stepper motor for its precision, but decided against it. In comparison to the servo motor, the stepper motor is slower, and requires a “homing” sequence (or some other form of external encoder) in order to learn its position, which is less effective than the positional encoding of the servo.

Mechanism

For last year's competition, the robot used an "all wheel" steering system which, while getting the job done at the time, proved to be problematic. This year, we circumnavigated these issues by employing a simple "front wheel only" steering system instead.

Based on previous experience, we knew from the start that minimising the turning radius would be vital to the success of the robot. The "all wheel" system used in previous versions of the robot was simply not worth the complexity and the single points of failure it caused, and thus, another method was needed.

To accomplish this, the robot now uses a steering bar driven by the arm of the servo to manipulate the front wheel mounts, so that they can move left and right with a ± 45 degree angle of steering. The linkages are made using m1.5 bolts and nuts, the latter of which are secured with hot glue to prevent vibrations from losing them and adding "slop" to the mechanism.



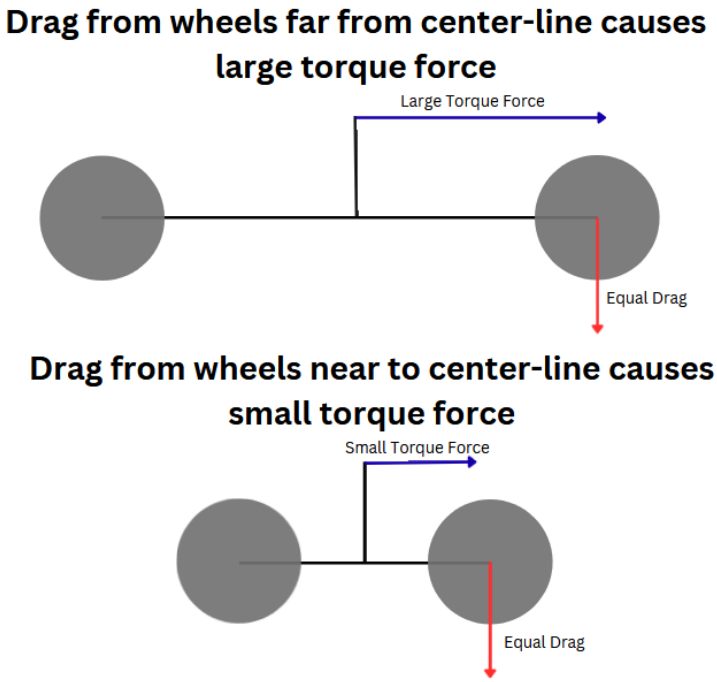
Simplified steering mechanism highlighted in red

Wheel Base

The final turning radius of a robot is directly proportional to the wheelbase of a robot, therefore this must be kept to an absolute minimum, the wheelbase of this robot is 70mm, which is tiny compared to the 110mm wheelbase of the previous iteration. This was achieved by shrinking the entire robot, and packing components as close together as possible, internally. Additionally, only components that were physically unable to be placed elsewhere were put between the front and back wheels.

Wheel Separation

Due to the lack of a differential drive, the back wheels spin at the same speed, resulting in one being “dragged” over the mat when the robot is turning. Therefore, the distance between these wheels needs to be kept to a minimum in order to minimise the moment of torque this creates.



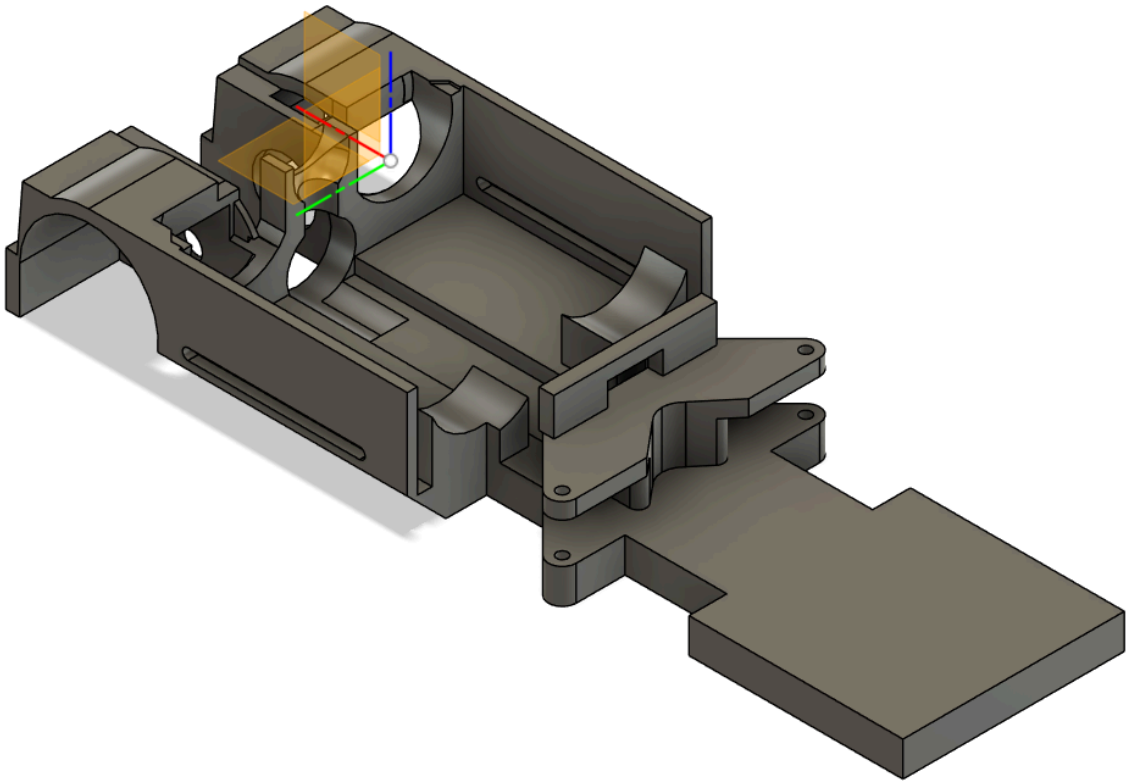
Conversely, the distance between the front wheels must be maximised relative to this, to maximise the torque they put on the robot to turn it, ensuring that the robot turns properly when commanded to do so.

CHASSIS

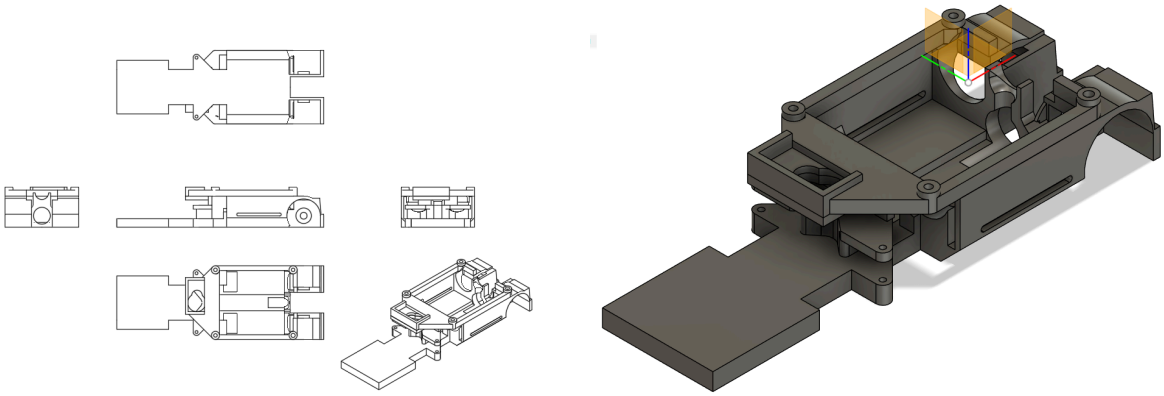
The chassis of our robot is entirely 3D printed from a mixture of PTEG and PTA filament, with PTEG (chosen for its resistance to becoming brittle, high strength and relative ease of printing) being used for the major components, and PTA (used for its high dimensional accuracy, and ease of printing) being used for smaller and non-critical components.

The chassis consists of two main layers: above and below the Raspberry Pi. The Raspberry Pi, spanning the full width of the robot and the majority of the usable length, essentially splits the robot in two.

The main unibody frame is printed as a single piece, simplifying the robot and reducing the required assembly. This frame houses the motor, motor driver, and the battery pack. It provides mounting points for the steering servo, LiDAR, voltage regulators, and the Raspberry Pi.



There are still some pieces that need to be assembled to create the main frame. These parts were separated to remove large overhangs that would otherwise need to be printed.



This frame houses all of the components occurring below the Raspberry Pi.

Above the pi, a second “frame” is present, connected to the rest of the vehicle by a series of pins, allowing for toolless removal and easy access to the components below. This upper-frame provides mounting points for the camera, gyroscope and other associated components.

Based on past experience, we decided that we needed to keep the size of the frame to a minimum, particularly in width, in order to allow for the maximum tolerance for error between the robot and the obstacles.

The components used for the chassis were printed using an ender 3 V2 Neo and an Ender 3 S1 Pro, with the following settings:

Parameter	**Value**
Wall Width	0.8
Print Speed	60mm/s
Infill	20%
Top/bottom thickness	0.5mm
Printing Temperature	200C-235C

Build Plate Adhesion	Skirt
Support	Tree

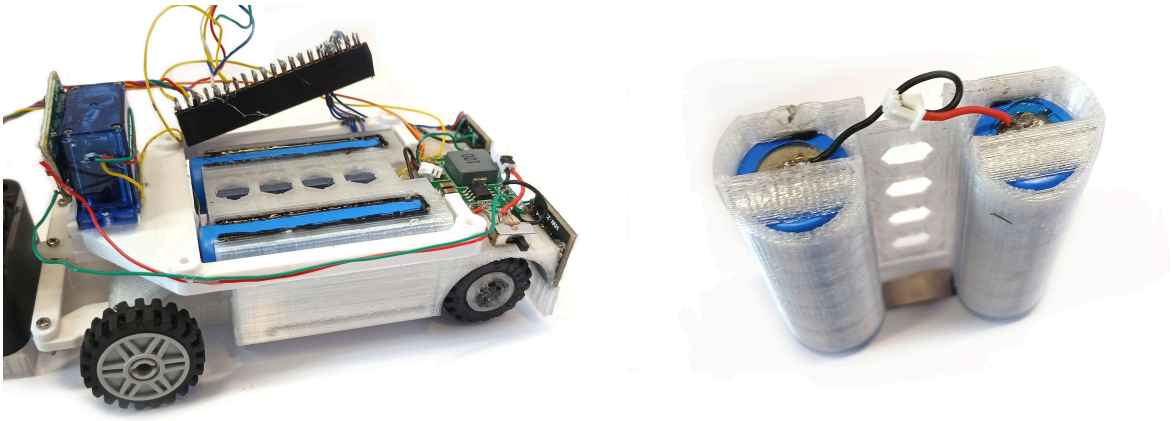
Power and Sense Management

BATTERY PACK

This version of the robot uses two batteries: an internal battery and an external battery. Due to the internal battery lasting about 45 minutes, the robot is commonly connected to an external battery pack while writing code. In our case, this is a 5000Mah 12V LiPo battery, but any large battery or power supply is suitable, provided it can produce the necessary current.

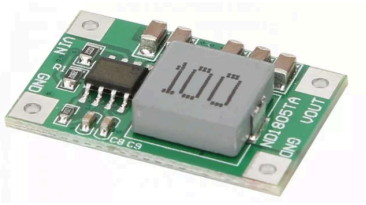
For the internal battery, we are using a custom battery pack assembled from 2 x 14500 Lithium Ion (Li-on) battery cells, and wired with a 1mm pitch JST XH connector. This gives us 1000mah of capacity at a nominal voltage of 7.2V. These batteries were chosen for their small size and high energy density.

As a result of the custom shape and connections of our internal battery pack, we had no way to charge these batteries safely using a commercially available charger. Therefore, a charging “frame” needed to be built. This is able to adapt the battery to the typical XT-60 and JST XH connectors of a Lithium-Ion Polymer (LiPo) battery. This allows the battery to be safely charged using a 2 cell LiPo charger.



Battery Pack installed in the robot, and standalone

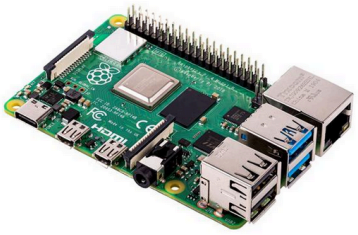
VOLTAGE REGULATION



We used 2 x 5A fixed 5V buck converters for voltage regulation. These drop the 7.2V battery down to the 5V that is used by the remainder of the robot.

Initially there was only a single voltage regulator, but although theoretically possible, the Raspberry Pi and servo sharing a power supply made the servo act erratically. Therefore, we split the power circuits.

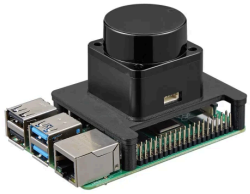
RASPBERRY PI



All the sensors are managed by a Raspberry Pi 4B running Raspbian, which we chose because of its processing power and ability to run multiple processes in parallel. Each sensor is assigned a process in code to ingest and process raw data. This data is then made available to the main program to be retrieved as needed.

This is a far simpler set-up than our previous robot, which used multiple microcontrollers to manage each process (with one core being assigned one process, and data being passed between boards using an I2C bus). This ended up resting multiple single points of failure, where loose wires or an excess of EMI would cause the robot to fail.

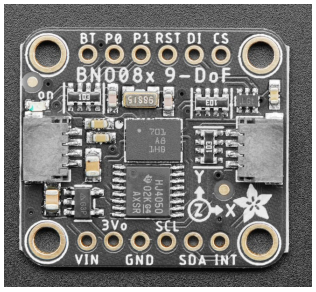
LiDAR



The sensors on our robot use LiDAR, which shoots out a beam of light at a known angle. Based on the time the reflected light takes to return, it can calculate the distance to that point on a potential object.

The sensor rotates 5 times per second, with 300 points per rotation. Therefore, you can determine the position of a point relative to the robot, based on its angle in relation to the robot and its distance from it. With enough of these points, you can assemble a dynamic “map” of the robot's surroundings, thereby constantly reading its position.

GYROSCOPE



This robot utilises a BNO08X gyroscope, mounted to an Adafruit development board. This board is capable of internal sensor fusion to output a highly accurate absolute angle. Communication with this board occurs over an I2C bus.

CAMERA

The camera used is a RunCam Thumb Pro. This is typically used as an action camera on first person view (FPV) drones.

This camera was chosen due to its 150 degree field of view, small and compact size, and its USB output.

This choice of camera does present a major issue though: When connected to USB, its default state is to act as a USB drive. It needs to be placed into “camera mode” by pressing the button on the camera. In this mode, it acts as a USB webcam.

Pressing this button proved to be a challenge, with our final solution being to use a relay and connections directly to the camera board to simulate the effects of a button press.

Obstacle Management

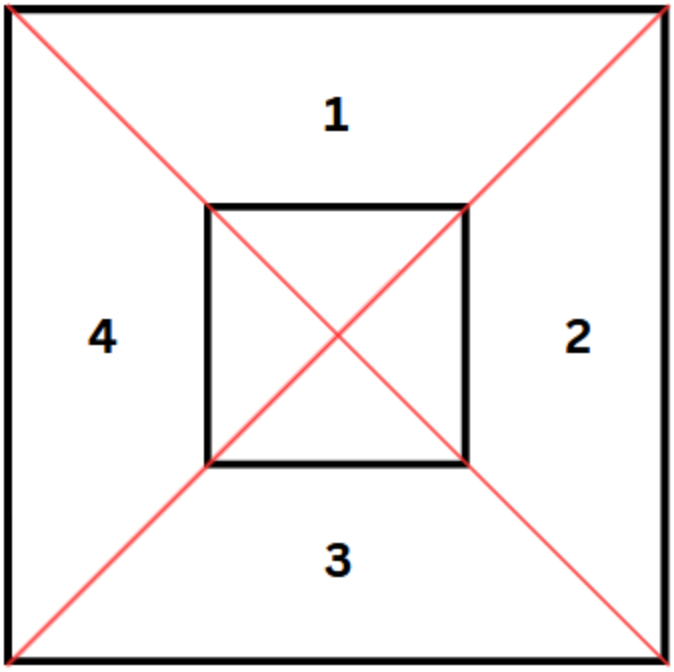
The robot runs the exact same program for both the open challenge and the obstacle challenge.

Initially, we attempted to use a Simultaneous Localisation and Mapping (SLAM) algorithm. This algorithm would have allowed the robot to map its surroundings, while using that same, potentially half built map, to navigate and make intelligent decisions. If we had been able to get this to work, it would have provided an ideal, self-recovering solution to the challenge. Unfortunately, it did not work, and thus we had to go back to the drawing board.

During our attempts at this algorithm, an additional challenge was noted: If the robot leant slightly to one side, the LiDAR sensor would look over the track boundaries and into whatever was beyond. This is suboptimal, as it would allow an object beyond the track boundaries to influence the navigation and decision making process of the robot.

To address these changes the LiDAR sensor was placed in front of the robot in a lower position. This resulted in a far greater acceptable margin of error (in terms of robot attitude) before the sensor potentially looks over the walls, or at the floor of the track. Although this limits the field of view that the LiDAR sensor has access to (to about 200 degrees as opposed to the 360 degrees previously possible, this drawback is massively outweighed by the ability to almost absolutely trust that the readings from the lidar are from a relevant object.

The current method of obstacle avoidance and navigation divides the track into 4 equal “segments”. The track is divided by two lines that run at 45 degrees between opposite corners of the outside wall. Segment 1 is defined as the segment that the robot starts in. The segments are then numbered increasingly depending on the order in which the robot will arrive at them. These segments are used as a navigation aid only, allowing the robot, assuming it knows which segment it is in, to pinpoint its exact position on the mat.



“Segmented” map

In our current algorithm, the robot scans the starting area upon boot attempting to look for walls using the RANSAC algorithm.

This should be a two way flow diagram:

1) Ideal scenario - three walls identified via RANSAC

If three walls are found (front, left and right), the robot can determine its starting point. There are only 6 possible open challenge starting points, and 2 obstacle challenge starting points. The code will “snap” the “starting” position of the robot to the middle nearest one, but keep its estimated position at the observed position.

The vehicle then attempts to determine the direction in which it will need to turn. This is done by comparing the lengths of the walls to the left and right, with the robot needing to turn in the direction of the shorter wall.

Sub flow diagram:

If the robot finds a wall that is significantly shorter than the other, the direction of turn is known, and the robot places a waypoint

Flow chart

Insufficient walls detected

Performance Videos

List of Materials

Code

LiDAR

This code is responsible for interpreting the serial data from the LiDAR module. The polar coordinates output by the LiDAR module are mapped and saved, allowing inferences and decisions to be made based on their layout.

We were not able to get the CRC functionality working with the LiDAR module, despite trying a large amount of CRC algorithms, including but not limited to:

CRC-32

CRC-32C

CRC-16/MODBUS

CRC-16/CCITT-FALSE

CRC-8/DALLAS-MAXIM

CRC-5/USB

CRC-16/USB

CRC-1

As a result of this, some filtering is required to remove points generated from obviously “bad” data. These filters are as follows:

- Remove any point beyond 3.5m of range (The maximum distance between any two points on the track is 3 meters).
- Remove "lonely" points. Isolated points without a set number of other points within a specific radius.

Gyroscope

[code snippet]

This code constantly polls the current angle of the gyroscope using the I2C functionality of the Raspberry Pi. It is responsible for the initial calibration of the gyroscope upon device startup.

Servo

[code snippet]

This code continuously adjusts the servo to keep the true angle of the vehicle as measured by the gyroscope, as close as possible to a target angle which is set programmatically. This is done continuously through the use of a simple proportional function.

Motor

[code snippet]

A target rpm is passed into this program. This program then adjusts the motor power output up and down in an attempt to maintain this RPM constantly. This is done to attempt to maintain a constant speed, while certain actions (such as turning the robot sharply) may require more or less power to perform at said speed in comparison to normal operations.

Navigation

[code snippet]

The details of this Navigation program are covered in the "Obstacle Management" section.

Simulation

[bunch of links to the simulation programs]

Before any hardware was built, the theory and algorithms powering the robot needed to be tested. Therefore, we wrote a series of python programs to simulate the robot on the game board environment. These programs also allowed for experimentation with, as well as the tuning of, algorithms such as A* (pathfinding) and RANSAC (wall detection).

Unfortunately, there was a major flaw in all the simulation programs written to date. The simulation programs used an absolute reference position, whereas the robot uses a relative reference position, where it is always placed at the center. This means that the simulation programs did not provide a near “drag and drop” solution to software as we had hoped.